

CO1-AT1-CODING CHALLENGE

NAME: V Rakshana

SLOT: B

REGNO :192425137

COURSE CODE: CSA0911

QUESTION-2:

CODE:

```
public class Employee {  
    String name;  
  
    Employee(String name) {  
        this.name = name;}  
  
    double calculateSalary() {  
        return 0;}  
  
    public static void main(String[] args) {  
        Employee e1 = new PermanentEmployee("Ram", 30000);  
        Employee e2 = new ContractEmployee("Sam", 100, 200);  
        System.out.println(e1.name + " Salary: Rs." + e1.calculateSalary());  
        System.out.println(e2.name + " Salary: Rs." + e2.calculateSalary());}}  
  
class PermanentEmployee extends Employee {  
    double basic;  
  
    PermanentEmployee(String name, double basic) {  
        super(name);  
        this.basic = basic;}  
  
    @Override  
    double calculateSalary() {  
        return basic + (0.2 * basic);}}  
  
class ContractEmployee extends Employee {
```

```
int hours;

double rate;

ContractEmployee(String name, int hours, double rate) {

    super(name);

    this.hours = hours;

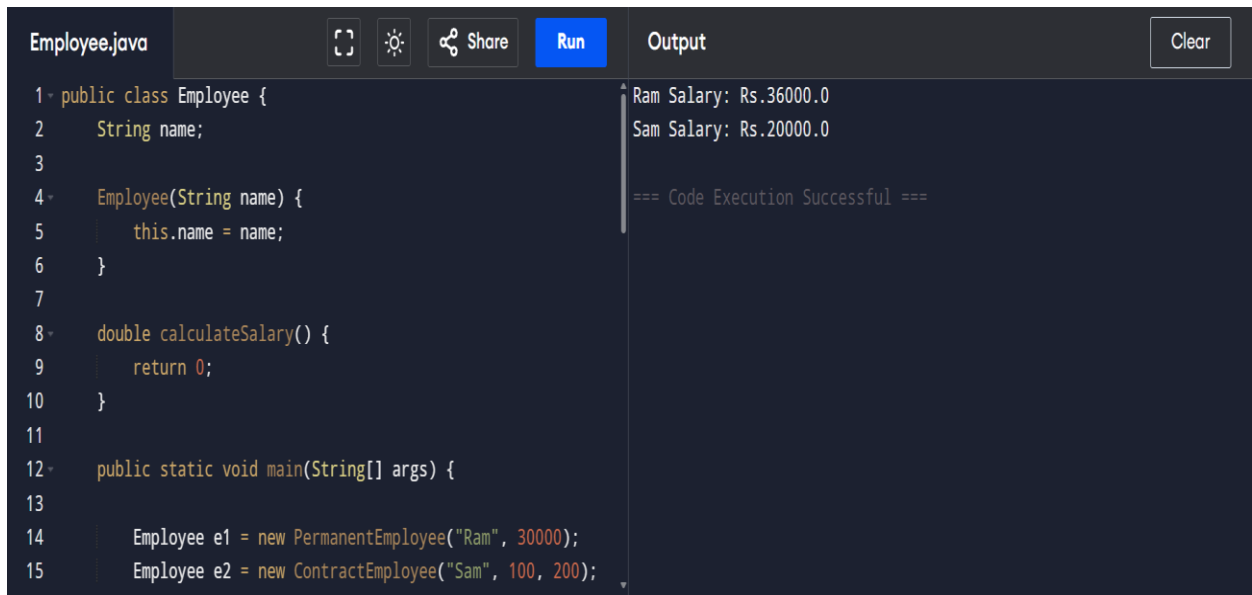
    this.rate = rate;}

@Override

double calculateSalary() {

    return hours * rate;}}
```

OUTPUT:

A screenshot of a Java IDE interface. The left pane shows the source code for 'Employee.java'. The code defines a base class 'Employee' with attributes 'name', 'hours', and 'rate', a constructor, and a 'calculateSalary()' method that returns 0. The 'main' method creates two objects: 'PermanentEmployee' for 'Ram' with 30000 hours and 'ContractEmployee' for 'Sam' with 100 hours and a rate of 200. The right pane shows the output of the program, which prints 'Ram Salary: Rs.36000.0' and 'Sam Salary: Rs.20000.0', followed by a success message. The interface includes buttons for 'Run', 'Share', and 'Clear'.